Linux Web Project

Step-by-step Apache deployment on Ubuntu 24.04

Project evidence and theory

This report documents the deployment demonstrated in the supplied screenshots: creating a project folder, writing a Bash script, making it executable, deploying an Infinite Loop web template, and viewing the completed site.

Prepared from the provided project screenshots

1. Aim and requirements

Aim: publish a static website through the Apache HTTP Server on Ubuntu 24.04. Apache serves the files placed in its document root, so visitors can open the site from a browser.

What is needed

An Ubuntu machine with internet access, a user with sudo privileges, Git, unzip, and Apache. The screenshot uses the KillerCoda Ubuntu playground; the same method works on a normal Ubuntu system.

Expected result: the Infinite Loop site loads from Apache after its HTML, CSS, JavaScript, and image files are copied into `/var/www/html`.

Key theory

Apache HTTP Server: a web-server program that listens for HTTP requests, normally on port 80. For a static site it maps a requested URL to files in its document root and returns those files to the browser.

Document root: the default Apache location on Ubuntu is `/var/www/html`. Putting an `index.html` file there makes it the default home page.

Bash script: a plain-text set of shell commands. The shebang `#!/usr/bin/env bash` selects Bash; execute permission allows it to run as `./proj.sh`.

> Safety note

The original screenshot removes the contents of Apache's document root. That is acceptable only in a temporary lab. On a real server, back up the current site or use a separate virtual host before replacing content.

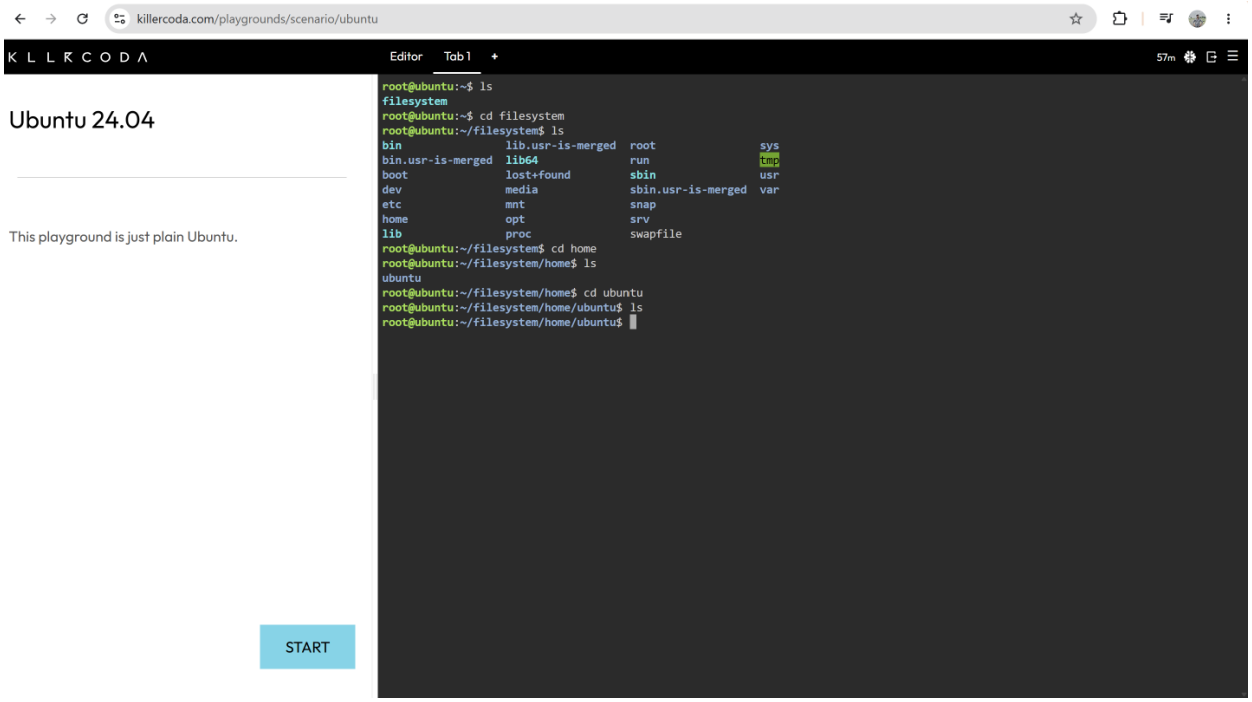


Figure 1. Starting in the Ubuntu playground and locating the user home directory.

2. Create the project script

In your home directory, create a folder and open the script. These match the commands shown in the screenshots.

```
mkdir project_linux
cd project_linux
vim proj.sh
```

In Vim, press `i` to enter insert mode, paste the script below, then press `Esc`, type `:wq`, and press Enter to save and exit.

```
#!/usr/bin/env bash
set -euo pipefail

# Update package information and install prerequisites
sudo apt update
sudo apt install -y apache2 git unzip

# Download the project and enter its directory
git clone https://github.com/thawaresameer715-cyber/Linux_Project_ApacheServer.git
cd Linux_Project_ApacheServer

# Extract the website files
unzip -o 2117_infinite_loop.zip -d site

# Replace the lab web root with the new site
sudo rm -rf /var/www/html/*
sudo cp -a site/2117_infinite_loop/. /var/www/html/

# Start Apache now and on future boots
sudo systemctl enable --now apache2
```

Why this version is safer and more reliable: `apt install` names the package manager, `-y` accepts installation prompts, `cd` ensures that the ZIP file is found after cloning, and `cp -a source/. destination/` copies hidden files as well as ordinary files.

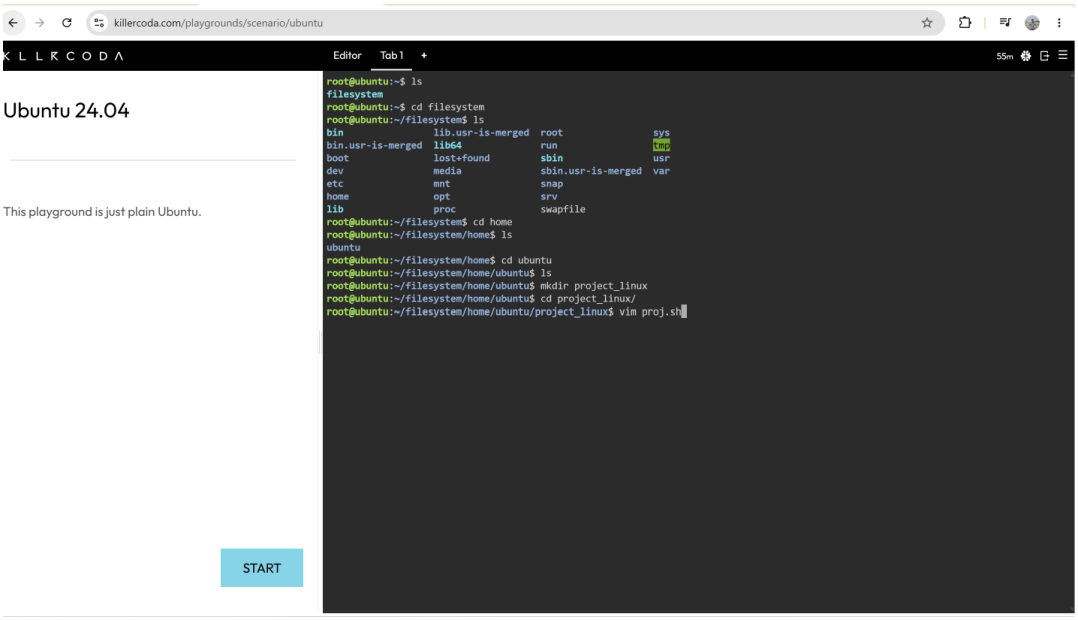


Figure 2. Project directory creation and opening proj.sh in Vim.

3. Make the script executable and run it

After saving the file, use the following commands:

```
ls -l proj.sh
chmod +x proj.sh
./proj.sh
```

Explanation: `ls -l` shows the file permissions. `chmod +x` adds the execute bit. Prefixing the filename with `./` tells Bash to run the script from the current folder.

During execution, enter your sudo password if requested. The script updates package information, installs Apache and supporting tools, downloads the template, extracts it, copies its files to the document root, and starts Apache.

Verification commands

```
systemctl status apache2 --no-pager
curl -I http://localhost
ls -la /var/www/html
```

A successful `curl -I` response should begin with an HTTP success status such as `HTTP/1.1 200 OK`. If Apache is not active, run `sudo systemctl restart apache2` and recheck its status.

Common fixes

Command not found: use `sudo apt install -y apache2`, not `sudo install apache2 -y`. **ZIP not found:** ensure you ran `cd Linux_Project_ApacheServer` first. **Permission denied:** rerun `chmod +x proj.sh`. **Default Apache page still appears:** confirm that the template's `index.html` is directly inside `/var/www/html`.

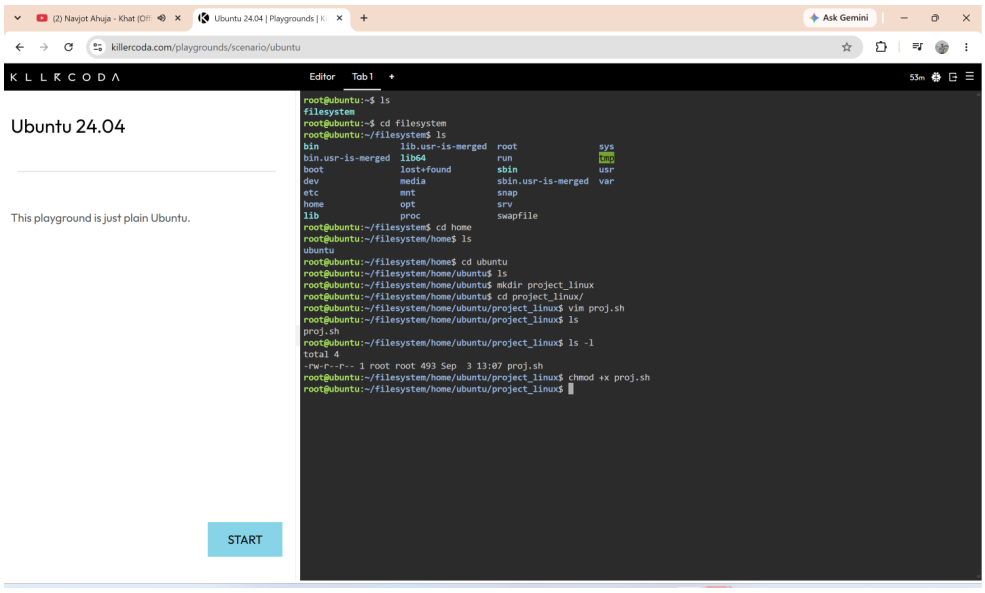


Figure 3. Confirming proj.sh exists and adding execute permission with chmod +x.

4. Deployment flow and result

The completed deployment follows this flow:

> Browser request -> Apache service -> /var/www/html/index.html -> linked CSS, JavaScript, images, and other static assets

When the browser requests the site, Apache reads the deployed files and sends them over HTTP. The browser then renders the page and requests the linked assets. This separation is useful because Apache handles delivery while the HTML template controls the visual design.

The final screenshot shows the Infinite Loop landing page rendered successfully, demonstrating that the template files were available through the web server.

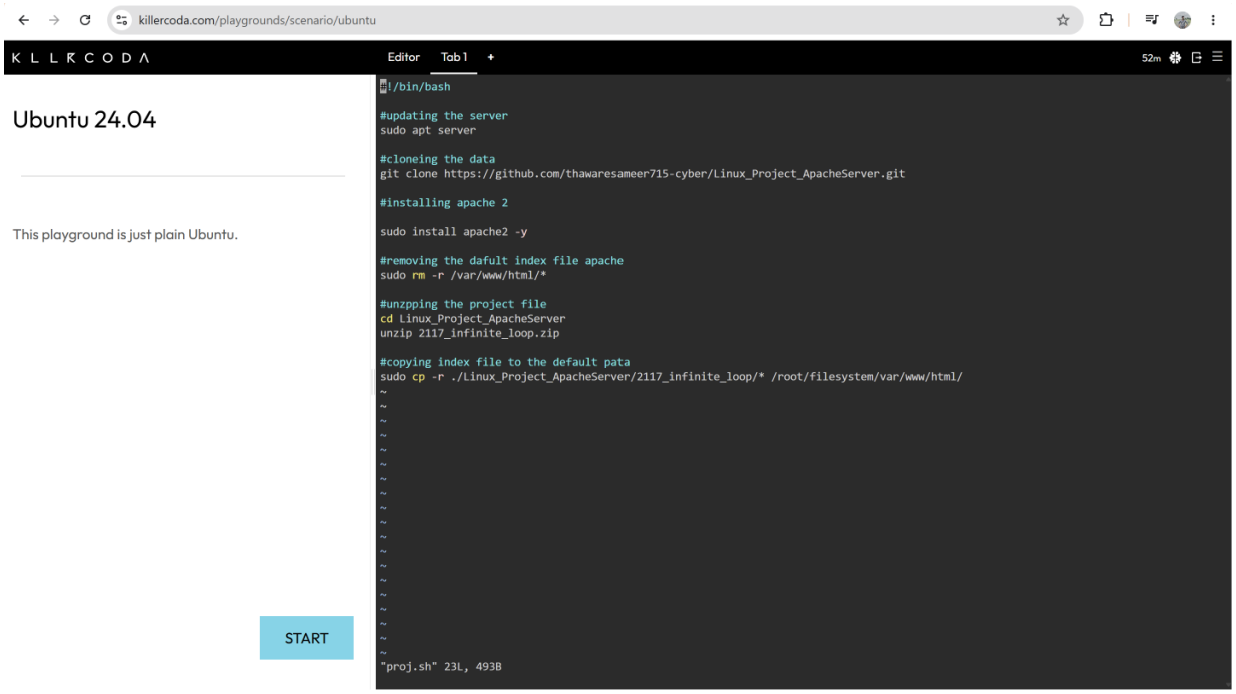


Figure 4. The Bash script as entered in Vim. The report's script on page 3 corrects its installation and working-directory details.

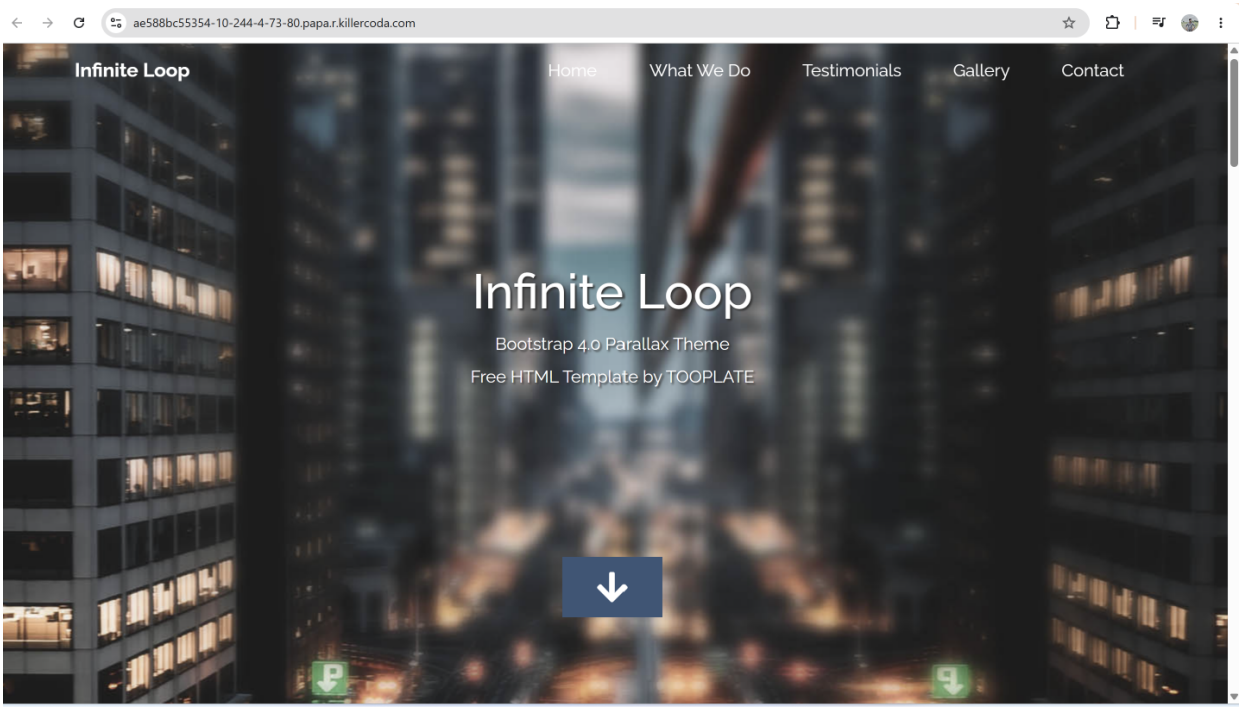


Figure 5. Successful deployment: the Infinite Loop website displayed in the browser.

Conclusion

The project demonstrates a complete static-site deployment: prepare the server, automate the installation with Bash, place website assets in Apache's document root, and verify the published page in a browser. This same pattern can be extended with a custom domain, HTTPS, and Apache virtual hosts for production use.